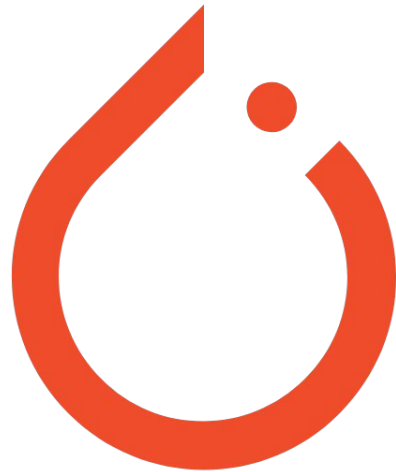


A Ground-Up Tour of **torch.compile**



Ayush Satyam

Associate MLE at  **Red Hat**

What is **torch.compile**?

torch.compile is a Just-In-Time (JIT) compiler that turns Python **PyTorch code** into highly optimized **executable kernel code** with a **single line decorator**

What **torch.compile** does?

PHASE 01 — SLOW

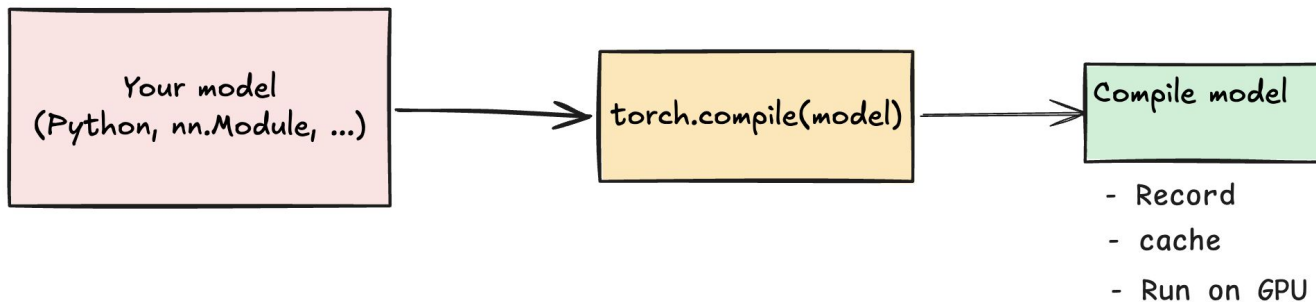
First Call JIT Compilation

Traces PyTorch code symbolically, compiles operations into optimized kernels, and caches them for future execution.

PHASE 02 — FAST

Subsequent Execution

Instantly reuses compiled kernels on every later call whose inputs match. A dynamic compiler watching Python at runtime.



Why **torch.compile** was needed?

Eager PyTorch was leaving performance on the table due to critical runtime overheads:

Overhead 01

Python Overhead

Fixed by fusing multiple operators into single execution **Kernels**, avoiding Python interpreter round trips.

Overhead 02

Memory Round Trips

Fixed by **Fusion**, minimizing read/write cycles to High Bandwidth Memory (HBM) to keep the GPU fed.

Overhead 03

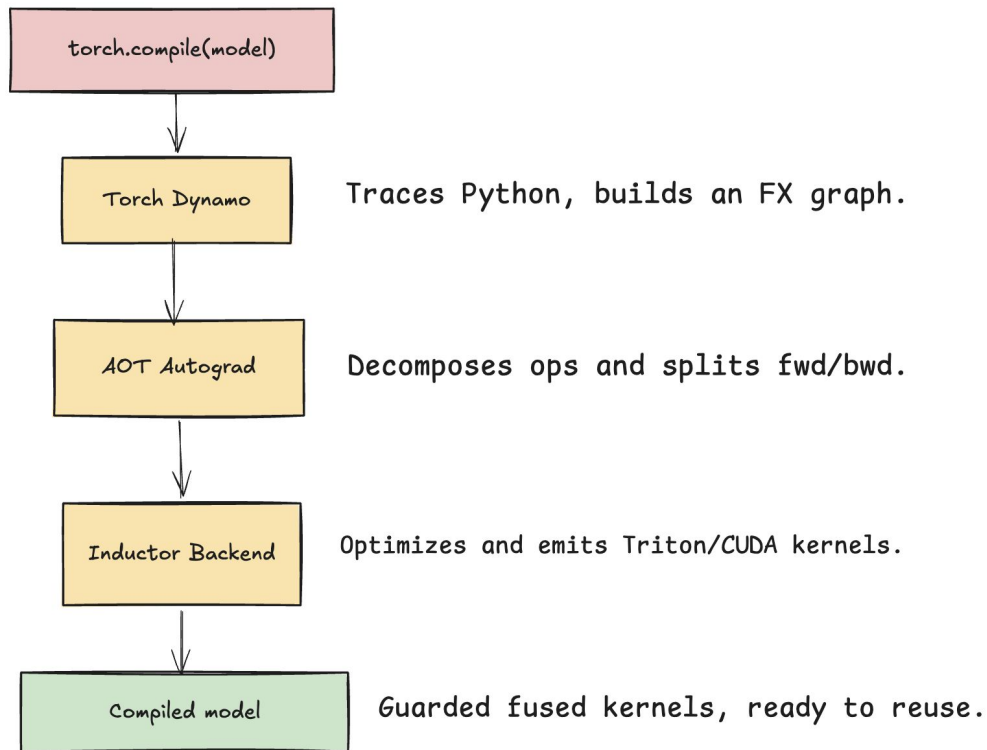
Kernel Launch Cost

Fixed by **Cudagraphs**, eliminating the per-call host-to-device CPU overhead of launching kernels.

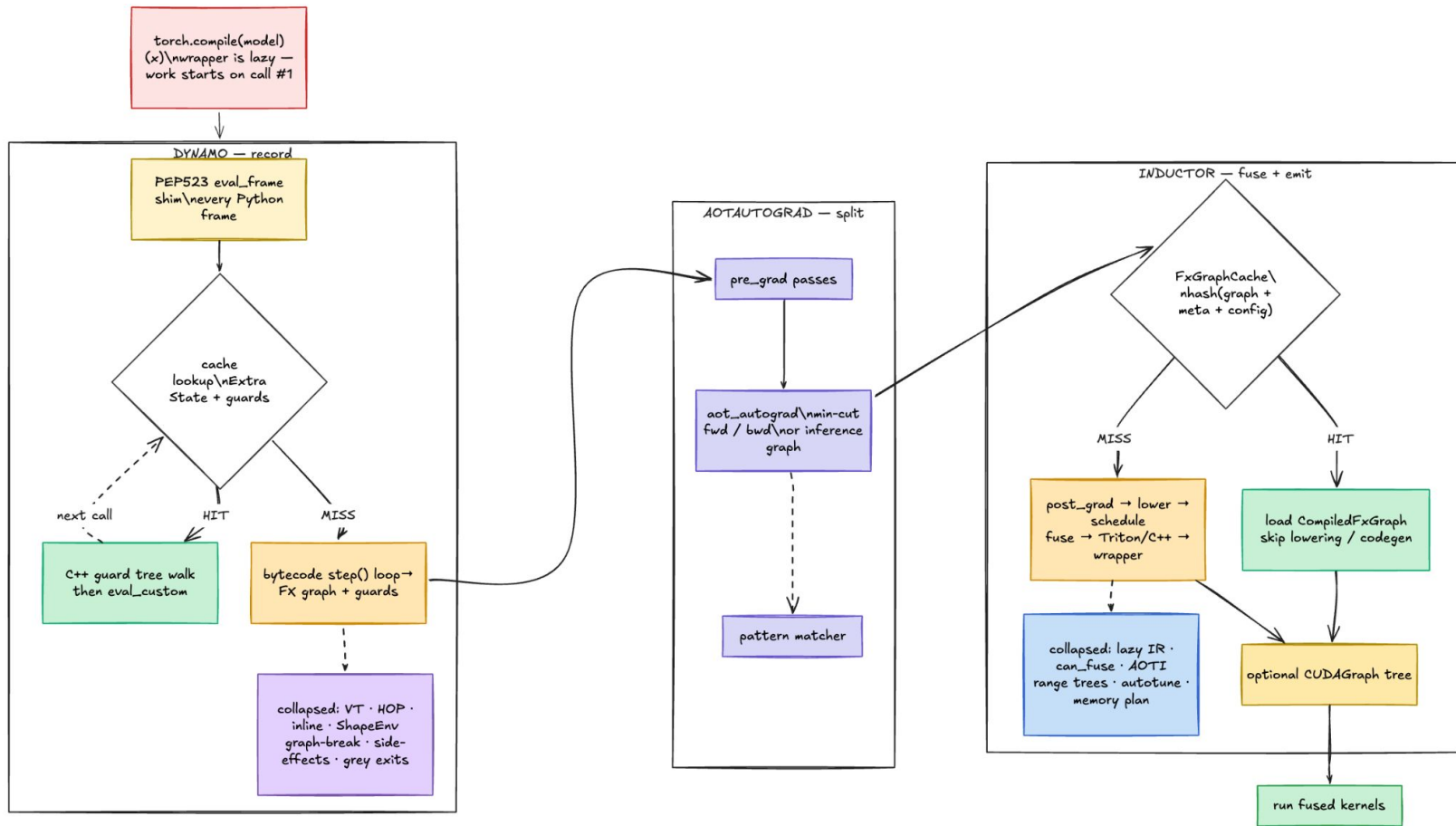
The **vocabulary** for torch.compile

Term	What it means?
nn.Module	A block of computation with weights. <code>forward()</code> is the math.
Op / kernel	A compute primitive (add, matmul) / the GPU code that runs it.
FX graph	PyTorch's internal script of ops, as data: nodes and edges.
Trace	Run the program <i>symbolically</i> to record op calls instead of computing.
Guard	A condition that must still hold for a compiled version to be valid. Fail => recompile. This is what makes JIT safe in dynamic Python.

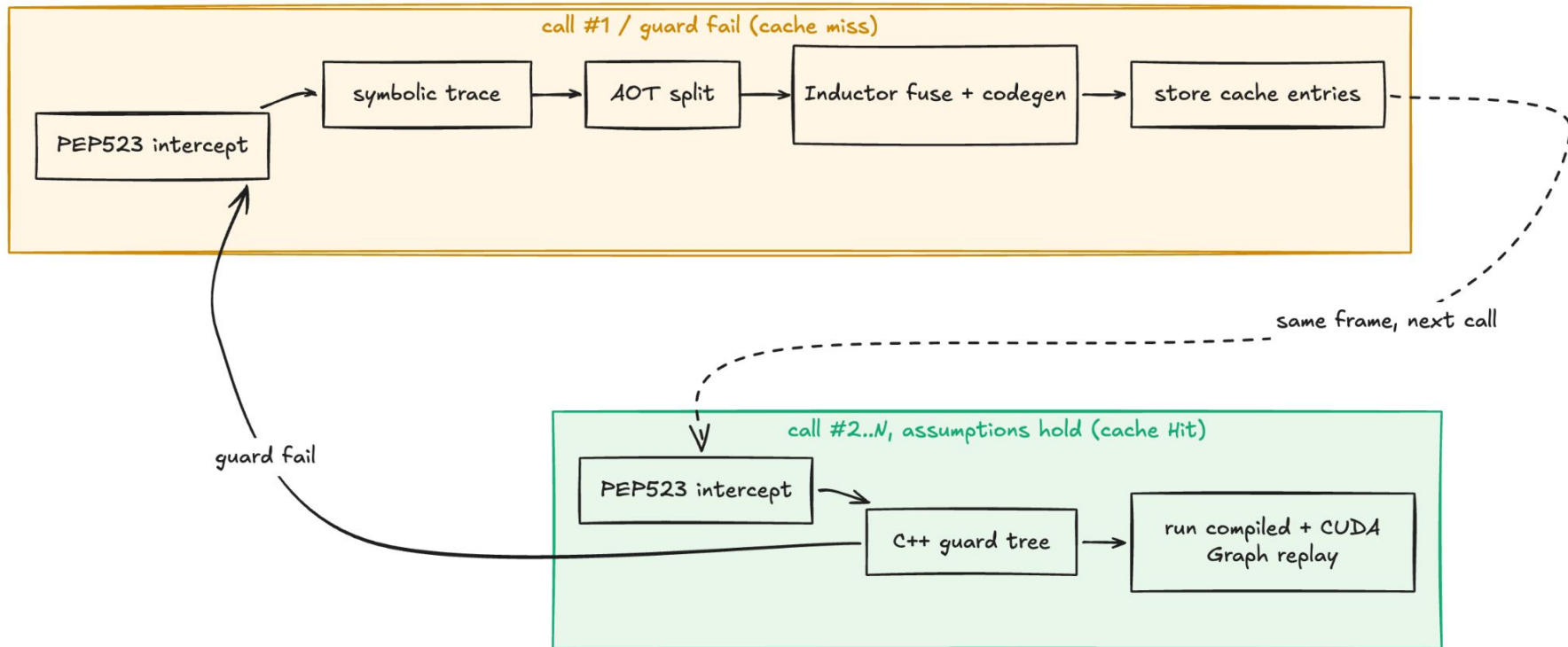
Control flow of `torch.compile`



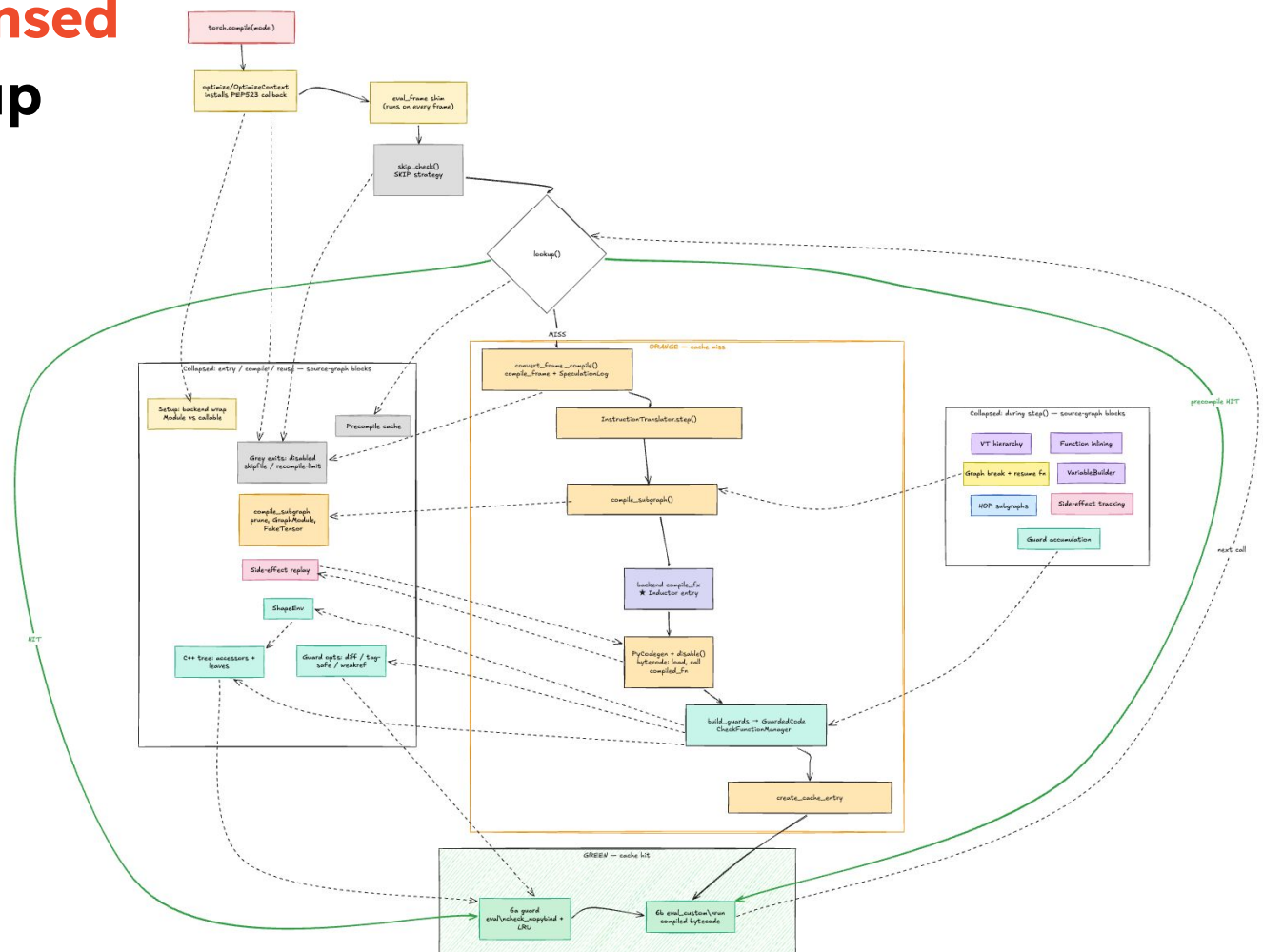
THE map: whole-system control flow in one picture



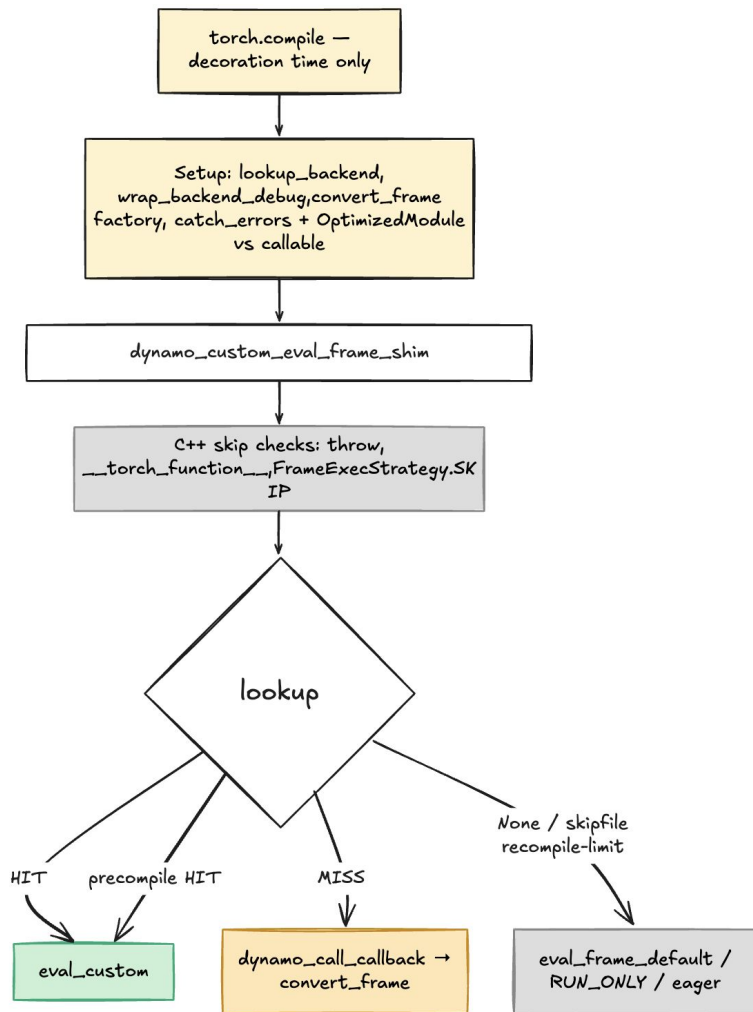
The runtime split that matters: miss vs hit



Dynamo **condensed** control flow map



PEP523, lookup and **frame** evaluation



Zooming on `step()` while loop

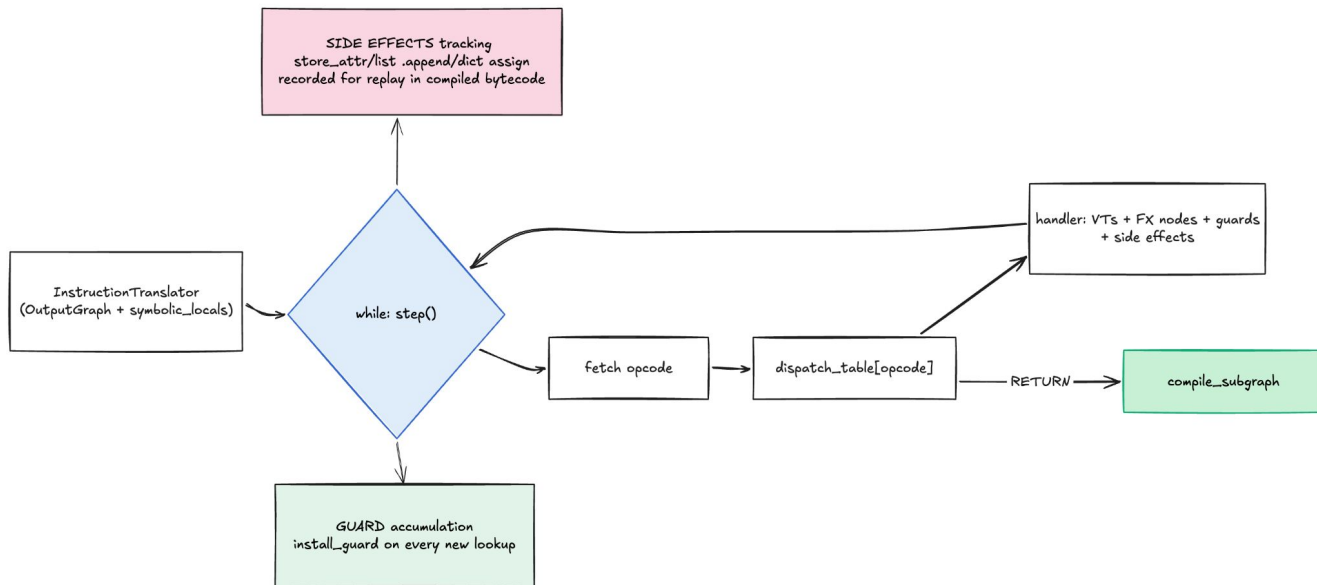
Execution Model

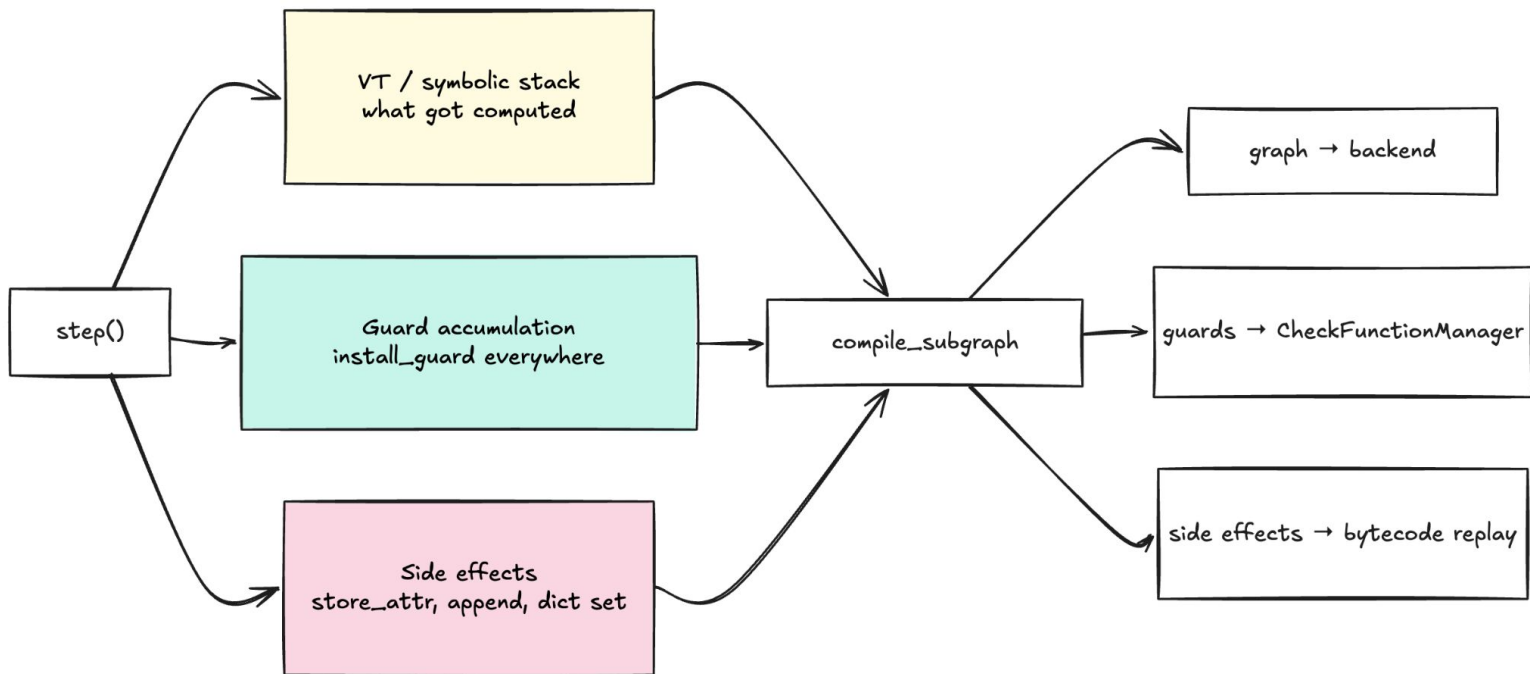
Parallel Execution

VT operations, Guard accumulation, and Side effects tracking fire in parallel for every single instruction.

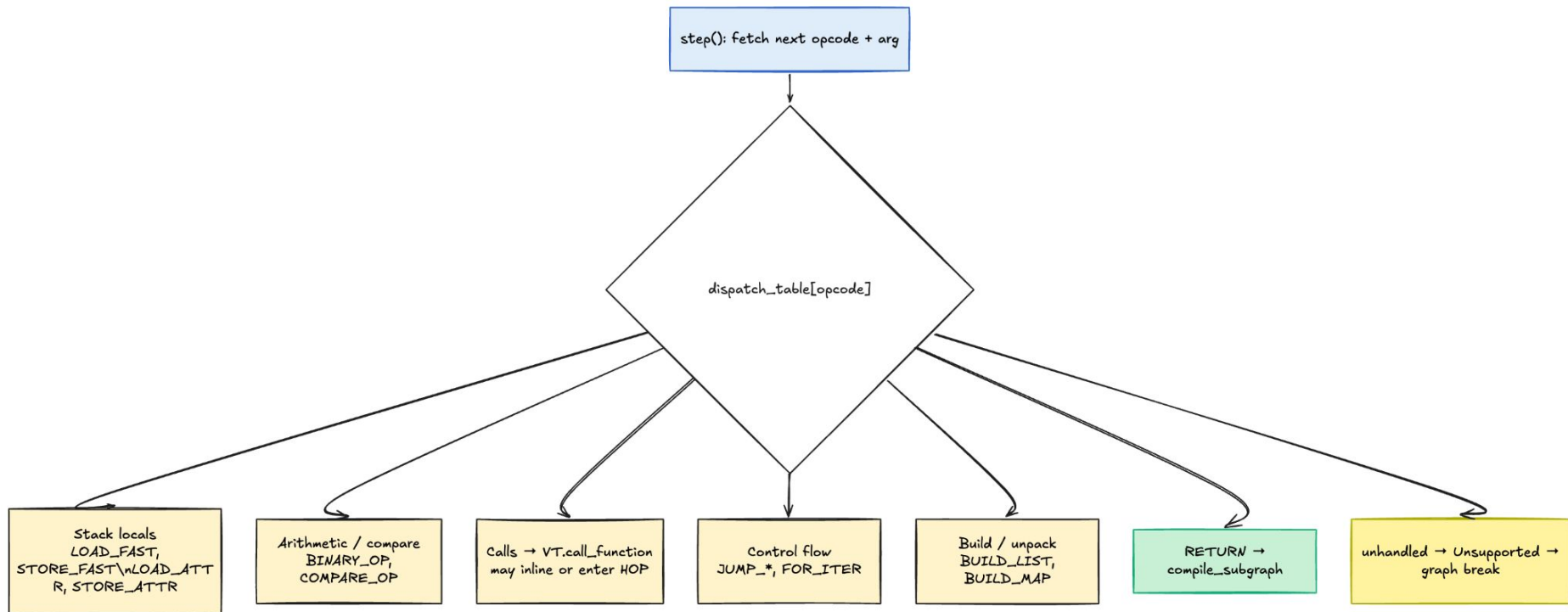
Unified Input

All three vital subsystems (including VTs) are fed continuously by the same underlying `step()` calls.

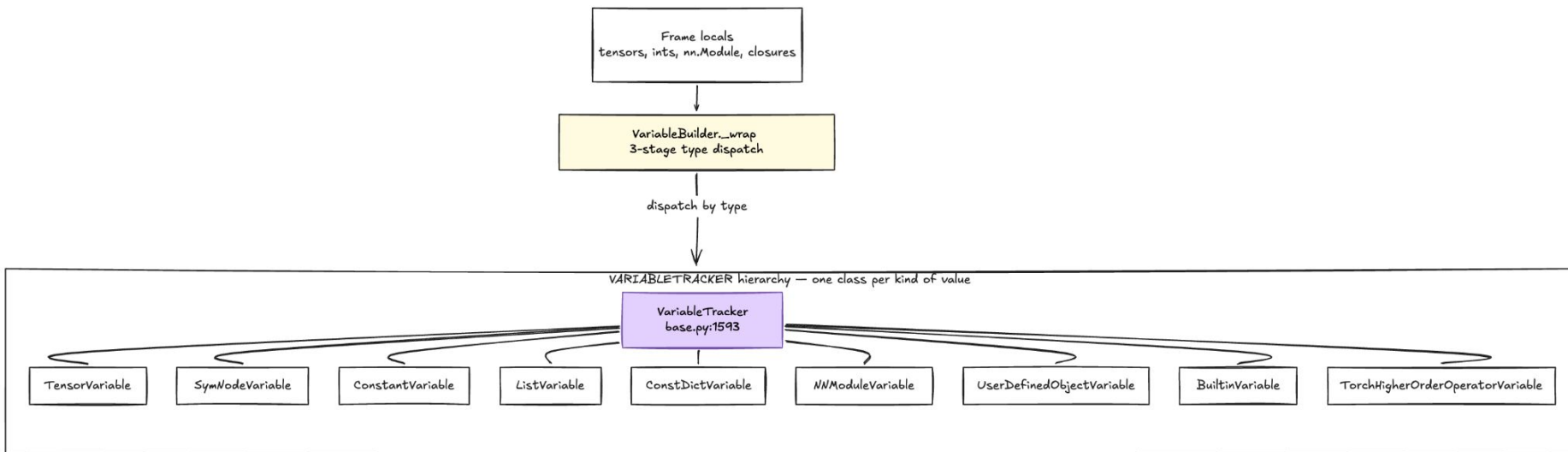




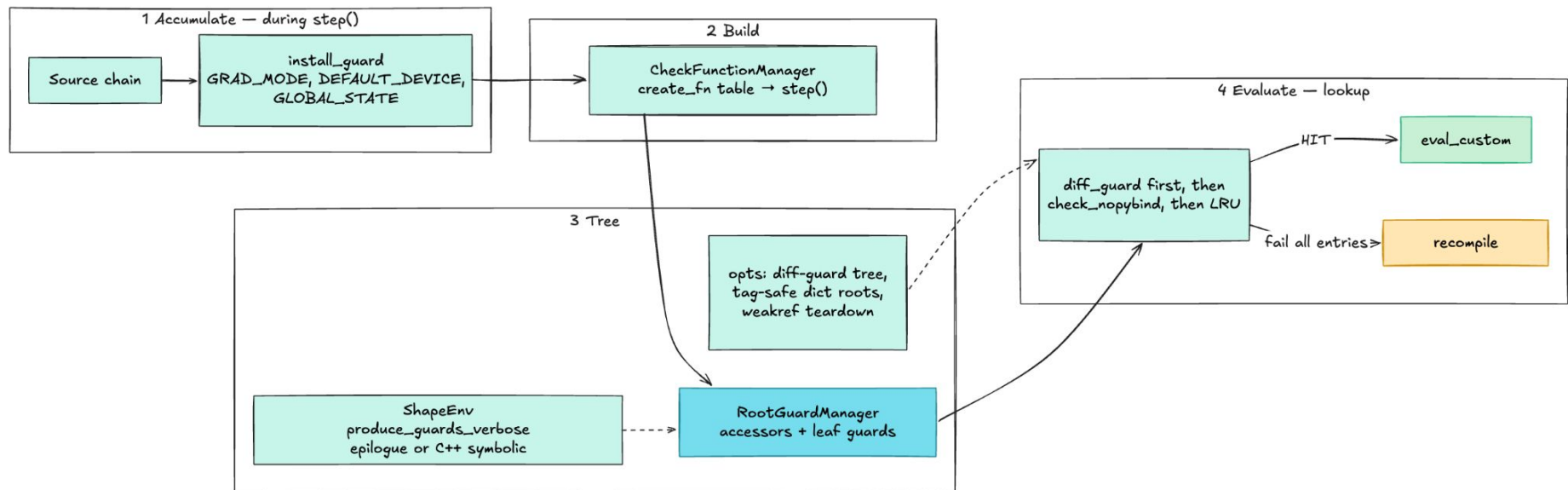
Lookup tables pulled off the Dynamo map



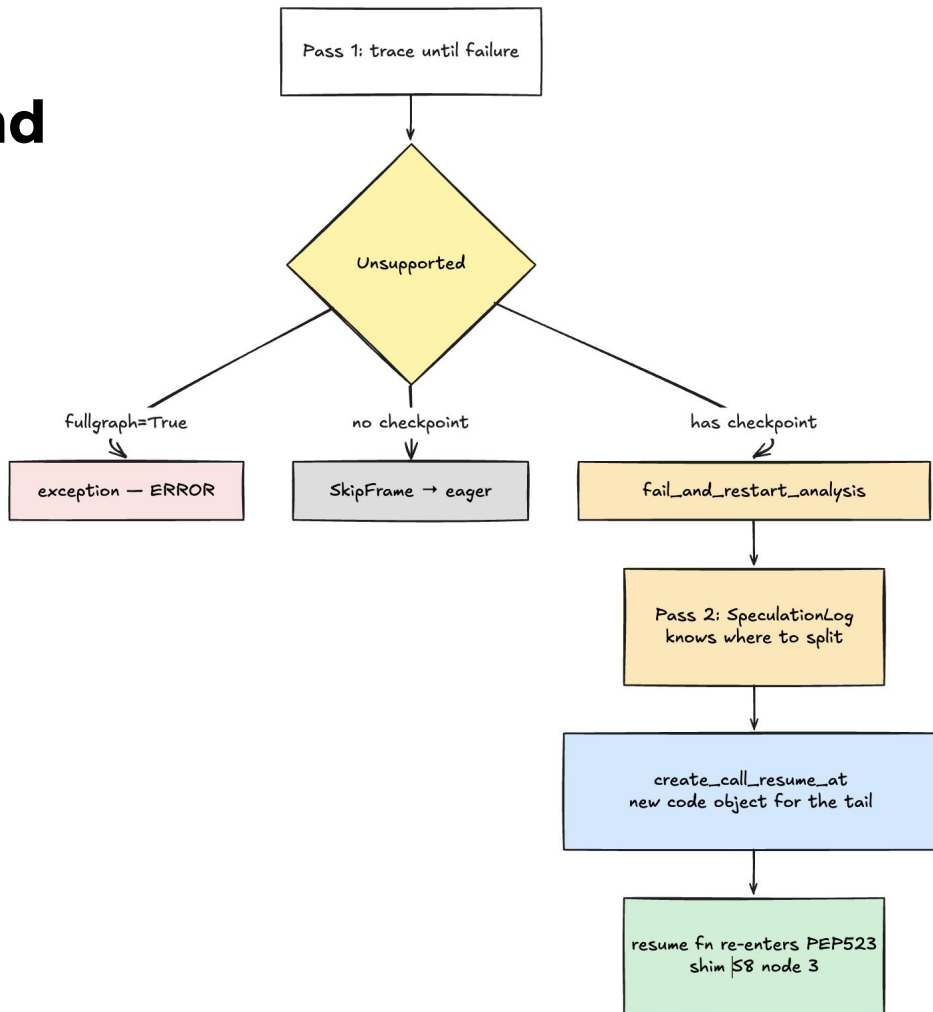
Zooming on the **VariableTracker** infrastructure



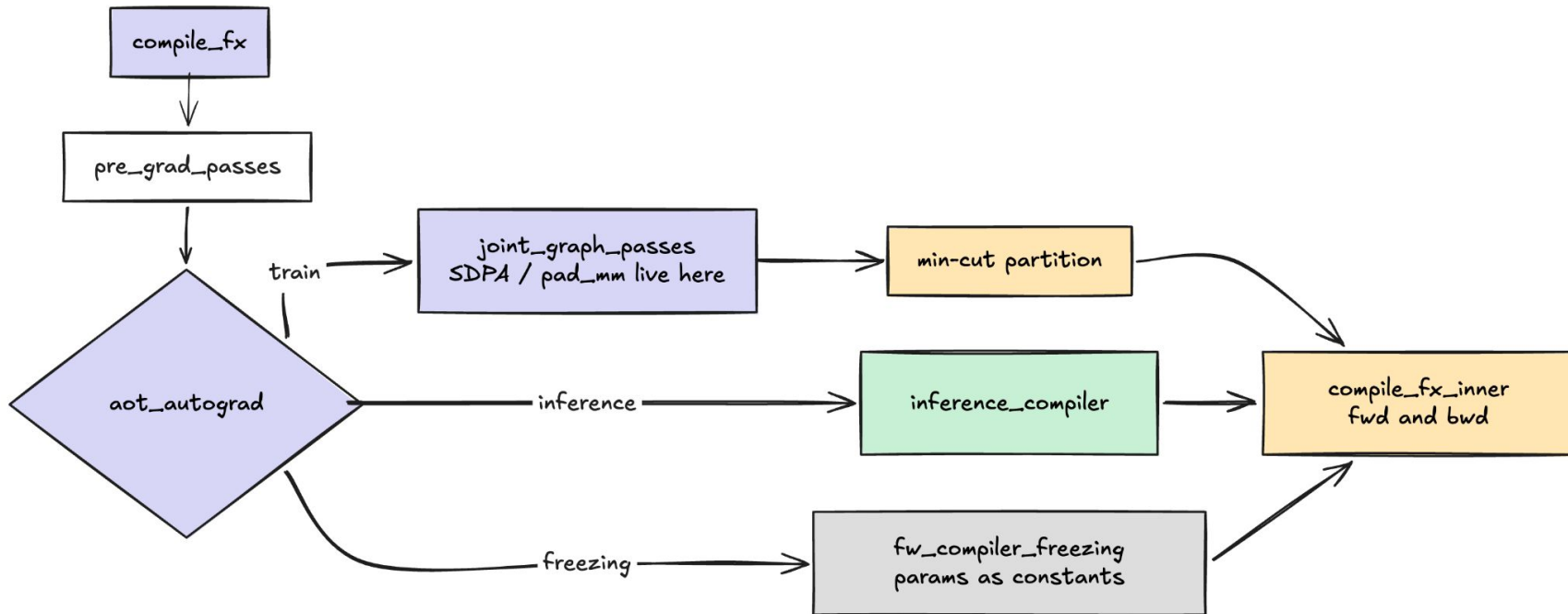
Full lifecycle of the **Guards** infrastructure in Dynamo



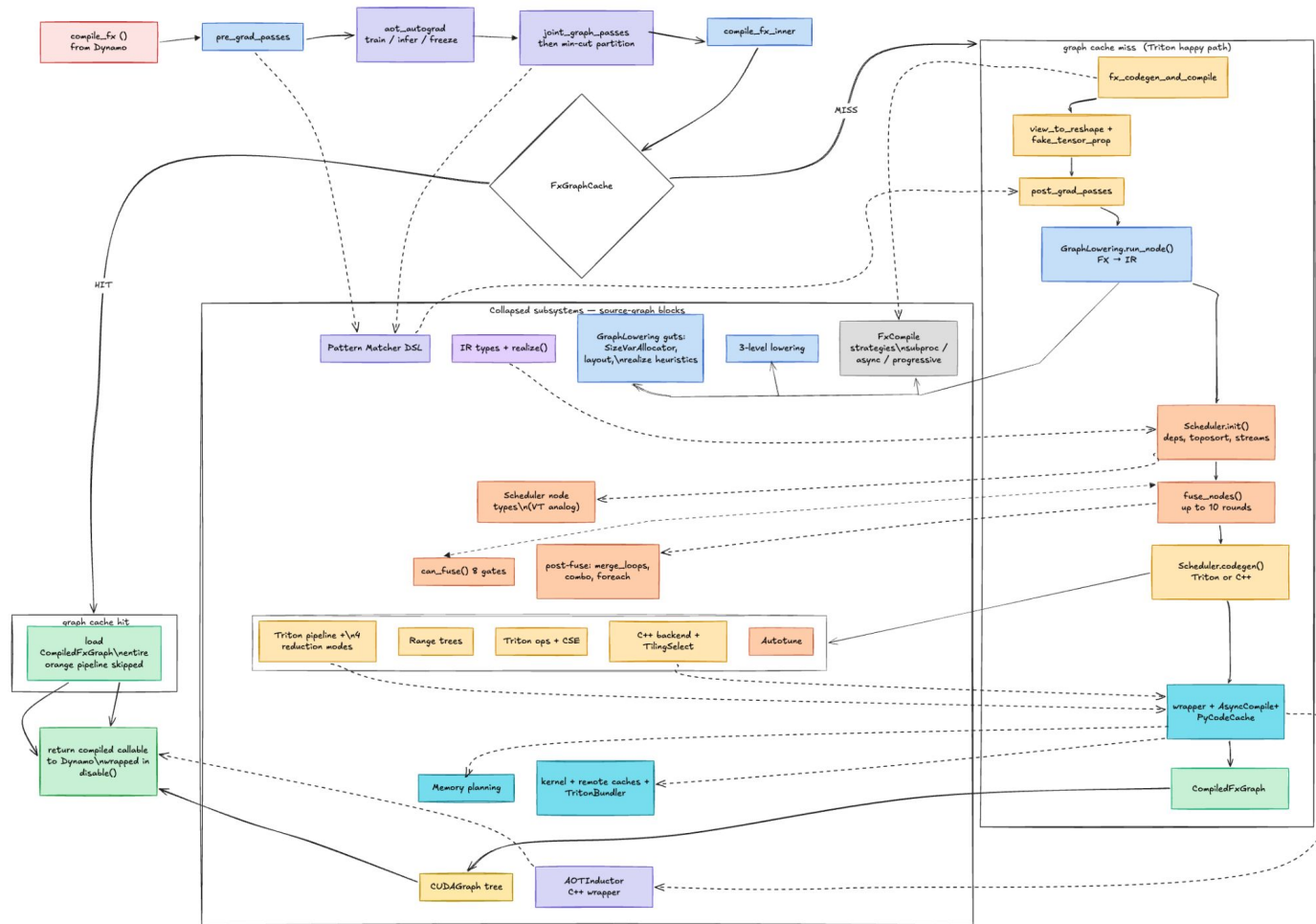
How **Graph-breaks** and recovery are handled



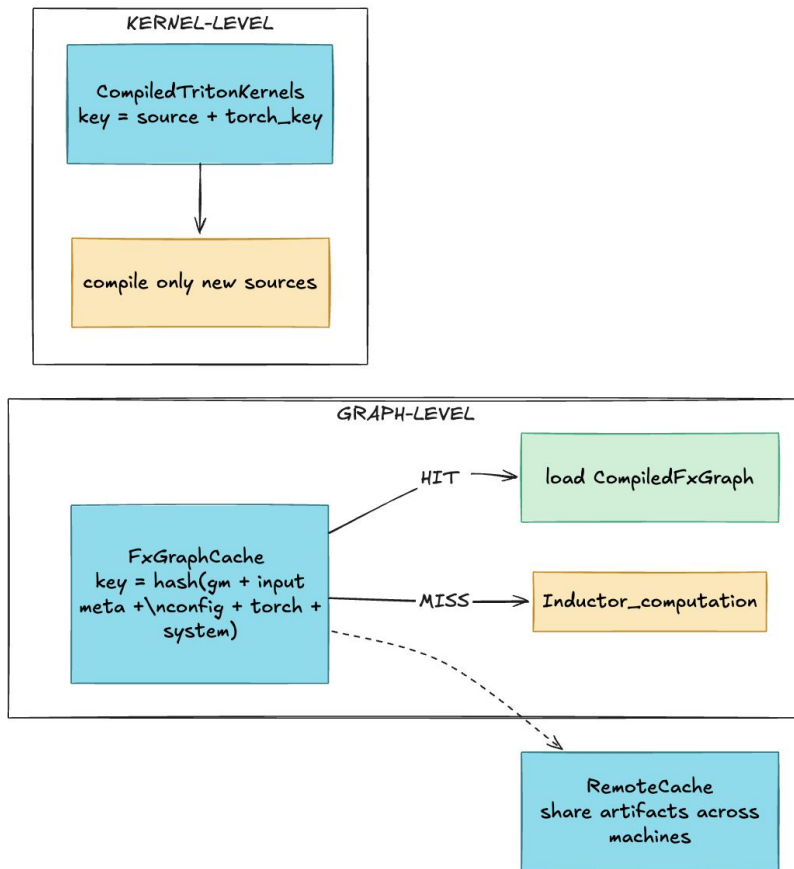
The **AOT-Autograd** bridge between Dynamo and Inductor



Inductor condensed control flow map



Inductor **Caching** mechanism



Zooming on **IR types** and lazy realize()

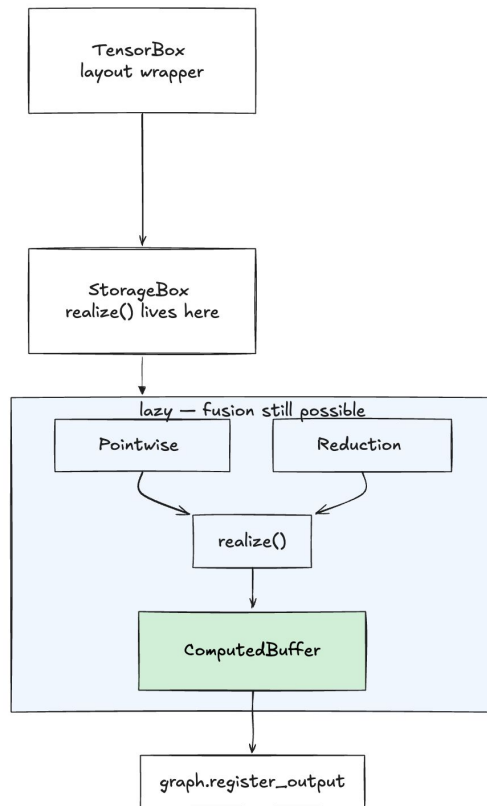
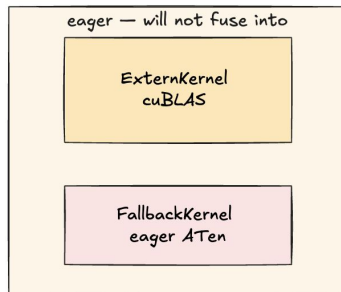
How Fusion Works

The reason fusion is possible is because **Pointwise/Reduction** etc. are descriptions, not buffers, until **realize()**.

`x.add(1).mul(2)` stays as two lazy Pointwise nodes until a heuristic materializes a **ComputedBuffer**. Fusion's job is to make that one buffer, with the other op inlined in registers.

Example Execution Flow

```
x.add(1)           # lazy Pointwise
x.add(1).mul(2)     # still two Pointwise, still no buffer
realize()           # one ComputedBuffer, mul inlined in registers
```

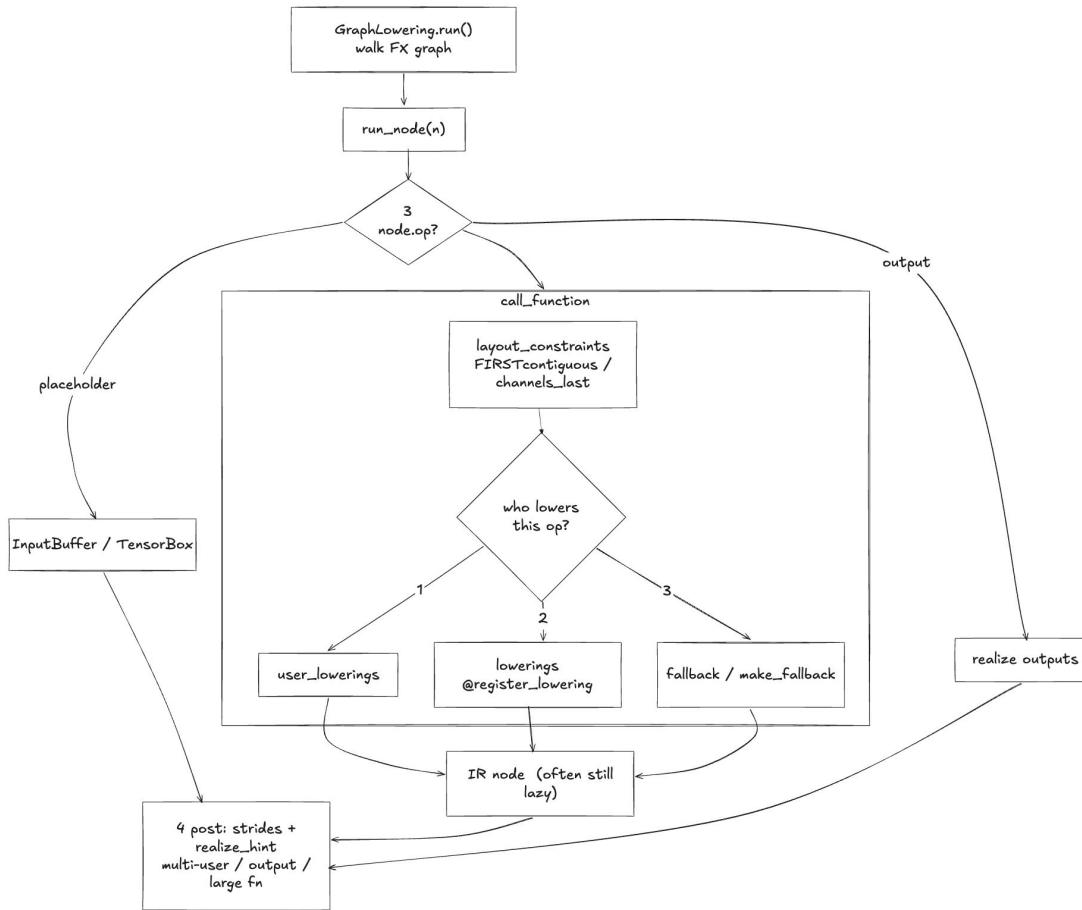


Zooming on **GraphLowering**, how we get the IR Node?

Core Concept

How Inductor IR is built:

GraphLowering walks the FX graph node by node and lowers each op to Inductor IR. **Most of that IR is still unevaluated**, which is the only reason fusion still has anything to work with.

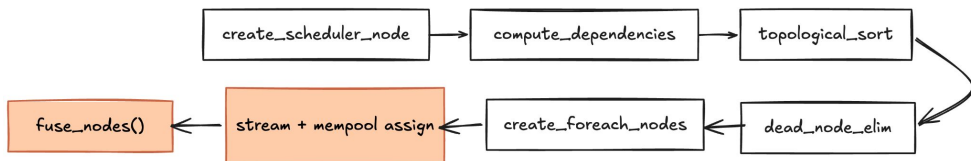


Zooming on Scheduler initialization

A bag of **lazy IR** becomes an **ordered DAG**. Fusion is not allowed to start until this finishes.

The collapsed scheduler node types node is the VT analog: `create_scheduler_node` picks a class, not a generic wrapper.

Scheduler node (Scheduler IR)	Wraps
<code>SchedulerNode</code>	one IR op
<code>ExternKernelSchedulerNode</code>	cuBLAS / oneDNN / etc.
<code>NopKernelSchedulerNode</code>	mark_run only
<code>ForeachKernelSchedulerNode</code>	grouped foreach
<code>FusedSchedulerNode</code>	result of <code>fuse_two_nodes</code>
<code>FusedMixOrderReductions</code> / <code>FusedNestedReductions</code>	reduction special cases



Analysing the fusion pipeline + the 8-gate `can_fuse()`

Looping & Fast Checks

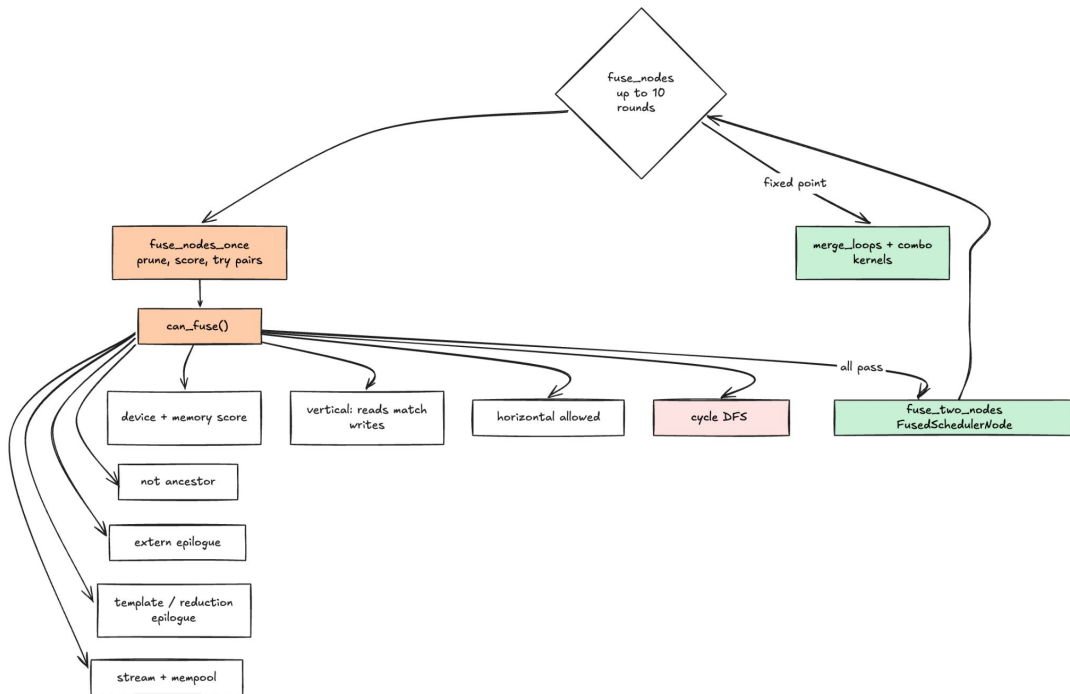
Cheap Filters First: Stream and mempool are checked immediately to eliminate illegal pairs quickly and cheaply.

Iterative Optimization: Fusion is an iterative loop, not a single pass. Merges expose new possibilities, repeating up to 10 rounds.

Strict Fusion Barriers

Cycle Detection: Ensures adjacent ops never merge if doing so closes a dependency loop, preserving separate kernels.

Score Evaluation: Device matching combined with a shared-data score acts as the final "legal but not worth it" filter.

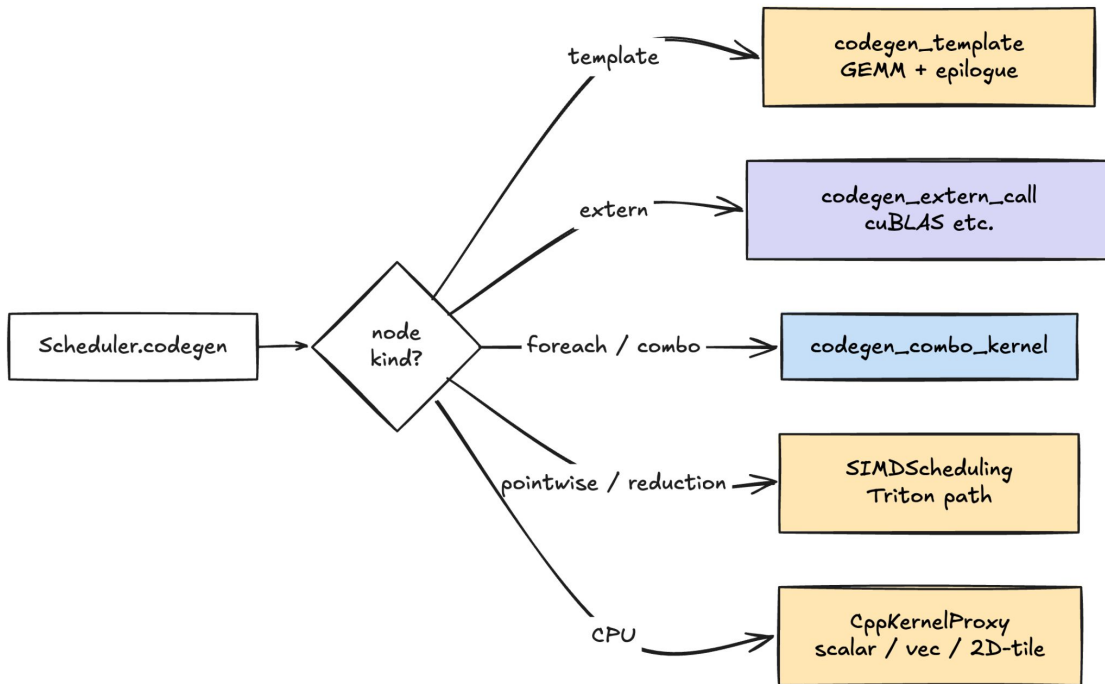


The Codegen dispatch

Execution & Fusion

The nn.Linear Example: Eager execution requires three launches, while compiled execution reduces this to two.

Triton Epilogue: GEMM remains a cuBLAS ExternKernel, but bias and activation fuse into a single Triton epilogue so intermediate results never hit HBM.



Dispatch & CPU Scheduling

Node Kind Dispatch: Dispatch is determined entirely by node kind (template, extern, combo, Triton, CPU) rather than the operation name.

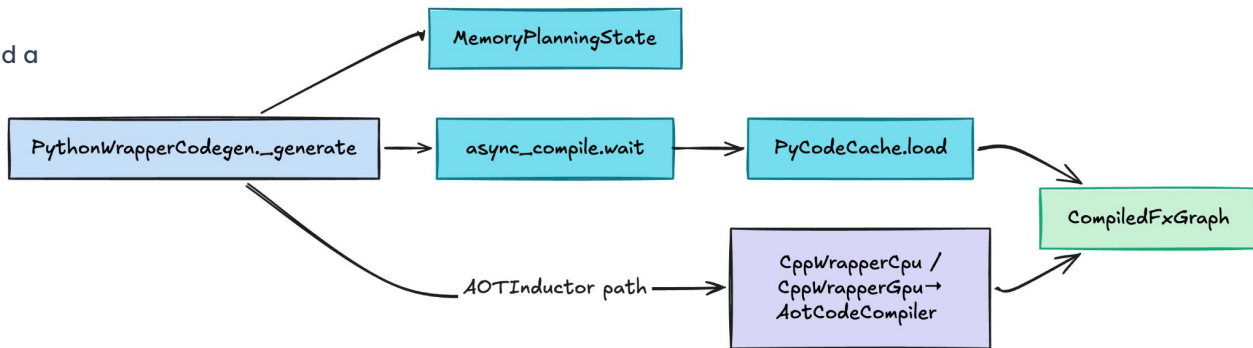
Unified Scheduler: CPU uses the exact same scheduler, where TilingSelect preserves either a scalar, vector, or 2D-tile layout.

Wrapper, **memory plan** and async compile

Compilation & Convergence

Triton & C++ Convergence: Triton and C++ converge here. First call should not wait for every kernel to finish compiling.

AsyncCompile: Employs a process pool for Triton and a thread pool for C++.



Memory & Alternatives

Memory Planning: Peak-memory optimization, not "malloc each node."

AOTInductor: The other wrapper; uses same scheduled units, C++ orchestration for export / AOT, and rejoins `CompiledFxGraph`.

Zooming on **CUDAGraph** tree

Graph-Based Execution

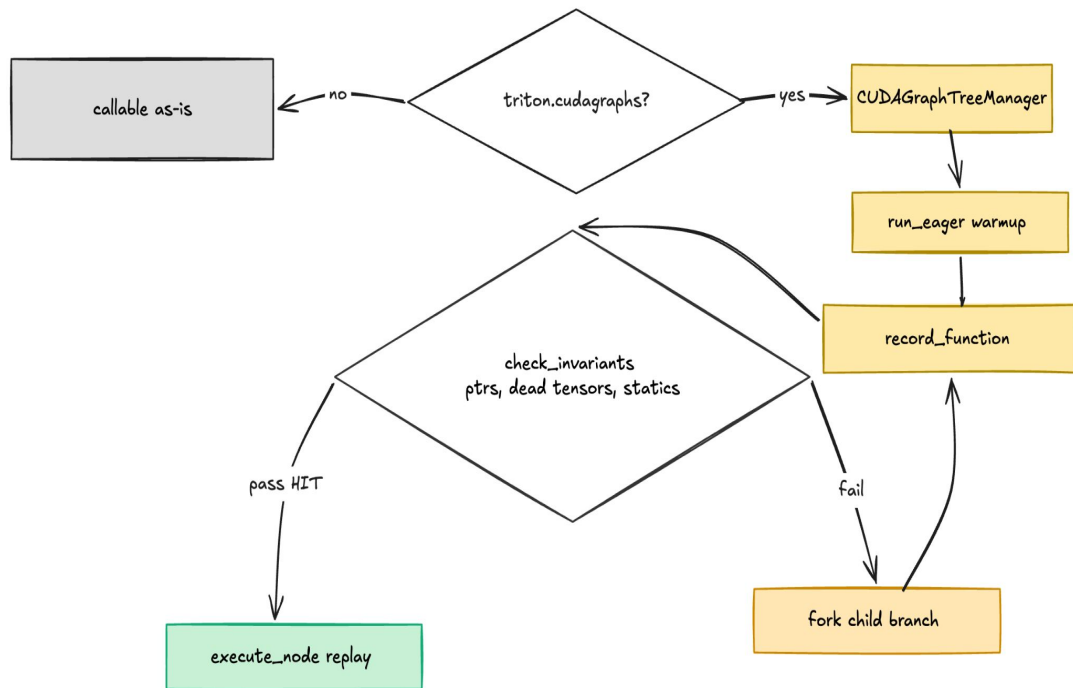
Fused kernels pay 5–20 μ s launch each. CUDA Graphs record once and replay with near-zero host work. The manager is structured as a tree, not a list.

Dynamic Branching

After replaying, different output-liveness can make a different subsequent recording valid. The runtime dynamically branches instead of aborting.

Lower-Level Caching

Operates on cache hit/miss logic one layer below Inductor's cache. It replays execution if invariants hold, otherwise it records a new child node.



Zooming on autotune

Launch Configurations

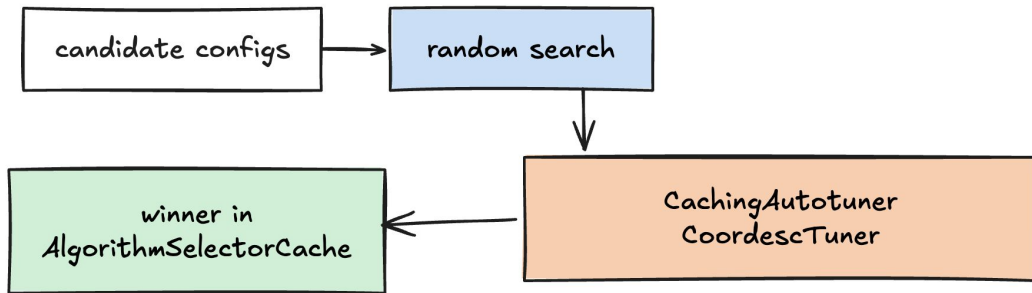
Template ops (GEMM/conv) have many possible launch configs. Finding the optimal one requires exploring a large search space.

Caching Autotuner

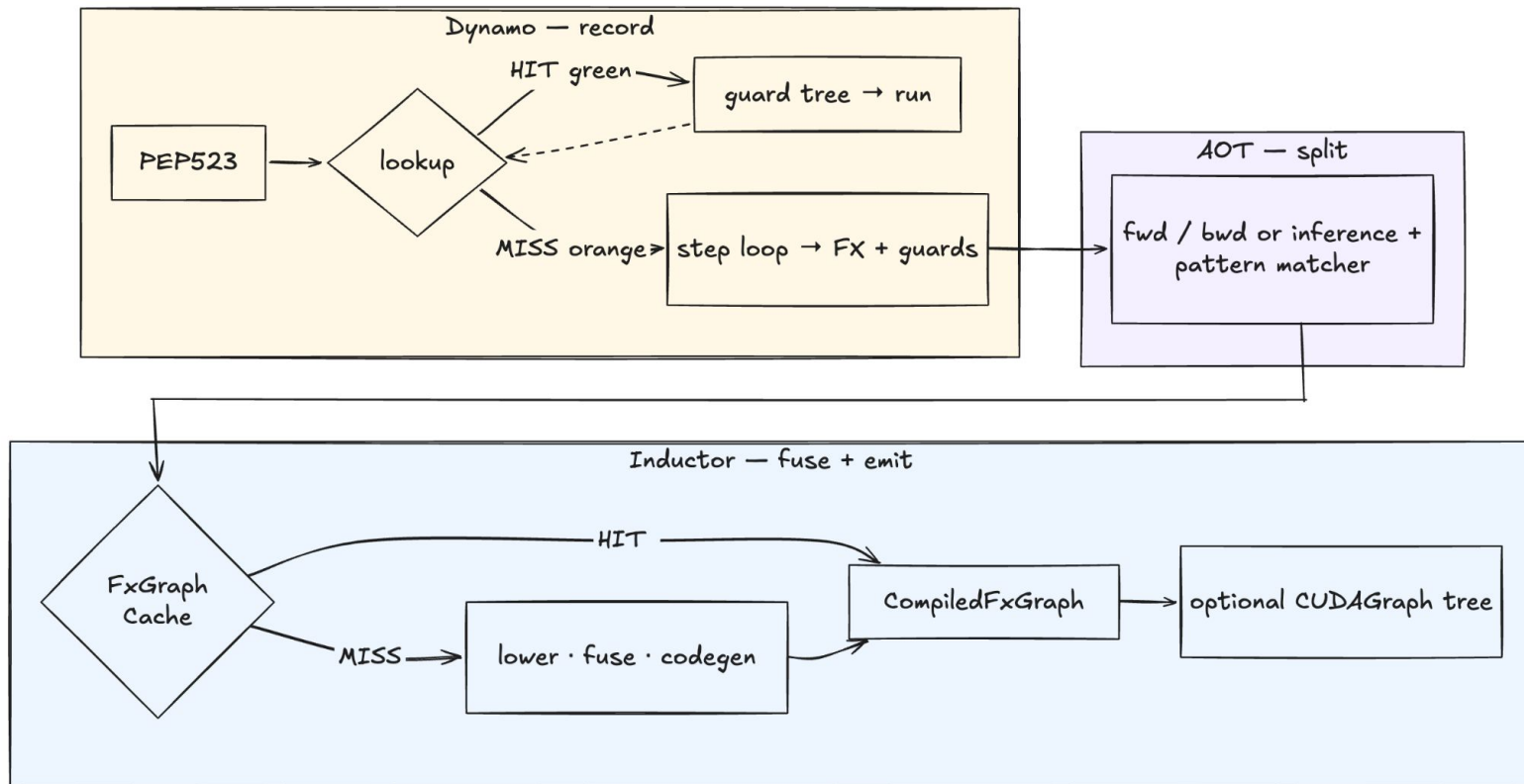
Random search or coordinate descent tuning is paid once. The winner is stored securely inside the AlgorithmSelectorCache.

Zero Runtime Overhead

Subsequent runs bypass the tuning phase completely, keeping the optimal configurations entirely off the critical per-call path.



A wide Recap of torch.compile



References:

- Pytorch Github [repo](#)
- My [Detailed ExcaliDraw illustration](#) for Dynamo and inductor

THANK YOU!